

Perfect. This is the level I'll maintain from now on.

---

# Java Debugging - Part 1 (Medium Depth)

## Topics Covered

1. Understand Debugging vs Guessing
2. Read Stack Traces from Top to Root Cause
3. Set Line Breakpoints in IDE

**Goal:** Learn how professional Java developers find bugs instead of randomly changing code.

---

## 1. Understand Debugging vs Guessing

### What is Debugging?

Debugging is the process of **finding the root cause of a bug before fixing it**.

Many beginners think debugging means fixing an error, but actually:

**Finding the cause is debugging.**

**Changing code is fixing.**

A good developer first understands **why** something is wrong, then decides **how** to fix it.

---

### What is Guessing?

Guessing means making random code changes hoping the problem disappears.

Example:

Suppose this code crashes.

```
String name = null;
```

```
System.out.println(name.length());
```

A beginner writes

```
if (name != null) {  
    System.out.println(name.length());  
}
```

The exception disappears.

Did we solve the problem?

**Not really.**

We still don't know **why** name became null.

Maybe:

- Database returned nothing
- API failed
- User input was missing
- Another method assigned null

The real bug still exists.

---

## Correct Debugging Approach

Ask questions.

```
Why is name null?
```

```
↓
```

```
Where was it assigned?
```

```
↓
```

```
Which method returned null?
```

```
↓
```

```
Should it ever be null?
```

↓

Fix the source.

Professional developers investigate before writing code.

---

## Coding Example 1

Wrong approach

```
public static void printUser(User user) {  
  
    if (user != null) {  
        System.out.println(user.getName());  
    }  
  
}
```

Correct thinking

Why is user null?

↓

Repository?

↓

Database?

↓

API?

↓

Wrong ID?

Fix the source instead of hiding the symptom.

---

## Coding Example 2

Suppose

```
int result = 10 / number;
```

Program crashes because

```
number = 0;
```

Wrong fix

```
result = 10 / (number + 1);
```

Correct fix

```
if (number == 0) {  
    throw new IllegalArgumentException("Number cannot be zero");  
}  
  
int result = 10 / number;
```

Now you're preventing invalid input instead of changing the calculation.

---

## Common Mistakes

- ✗ Adding `if(obj != null)` everywhere.
  - ✗ Changing multiple lines before testing.
  - ✗ Ignoring the actual exception.
  - ✗ Assuming instead of verifying.
-

## Best Practices

- ✓ Reproduce the bug first.
  - ✓ Understand the cause.
  - ✓ Make one change.
  - ✓ Test again.
  - ✓ Remove temporary debugging code after fixing.
- 

## Interview Points

Interviewers want to know whether you:

- Think logically
  - Read errors carefully
  - Solve root causes
  - Avoid random fixes
- 

## 2. Read Stack Traces from Top to Root Cause

One of the most valuable Java skills.

When an exception occurs, Java prints a **stack trace**.

A stack trace tells you:

- What happened
- Where it happened
- Which methods were called

Think of it as a roadmap to the bug.

---

### Example 1

```
public class Main {
```

```
public static void main(String[] args) {  
    String text = null;  
    System.out.println(text.length());  
}  
}
```

Output

```
Exception in thread "main"  
java.lang.NullPointerException  
at Main.main(Main.java:7)
```

Let's understand it.

```
java.lang.NullPointerException
```

Exception type.

```
Main.java:7
```

The exception occurred on line 7.

That is usually your first place to investigate.

---

## Example 2

```
public static int divide(int a, int b) {  
    return a / b;  
}
```

```
}  
  
public static void main(String[] args) {  
    divide(10, 0);  
}
```

Output

```
Exception in thread "main"  
  
java.lang.ArithmeticException: / by zero  
  
at Main.divide(Main.java:3)  
  
at Main.main(Main.java:9)
```

Notice something important.

The error is inside

```
divide()
```

not

```
main()
```

The stack trace shows the sequence of method calls.

---

## Reading a Stack Trace

Always read in this order.

### Step 1

Exception type

Example

```
NullPointerException
```

## Step 2

Exception message

Example

```
/ by zero
```

## Step 3

First line from **your code**

```
at Main.divide(Main.java:3)
```

That is usually where debugging starts.

---

## Real Project Example

Suppose Spring Boot prints

```
NullPointerException  
  
at UserService.getUser(UserService.java:45)  
  
at UserController.getUser(UserController.java:18)
```

Don't start with the controller.

Go to

```
UserService.java line 45
```



because that's where the exception actually occurred.

---

## Common Mistakes

- ✗ Reading only the last line.
  - ✗ Ignoring the exception message.
  - ✗ Looking at random classes.
  - ✗ Not checking line numbers.
- 

## Interview Points

A stack trace helps you answer:

- Which exception occurred?
  - Where did it occur?
  - Which method caused it?
- 

## 3. Set Line Breakpoints in IDE

A breakpoint tells the debugger:

▮ Pause execution here.

This allows you to inspect your program before it continues.

---

## Example

```
int a = 10;

int b = 20;

int sum = a + b;

System.out.println(sum);
```

Suppose you place a breakpoint on

```
int sum = a + b;
```

When debugging starts,  
execution pauses before this line.

Now you can inspect

```
a = 10  
  
b = 20
```

without writing any `println()` statements.

---

## Why Use Breakpoints?

Without debugger

```
System.out.println(a);  
System.out.println(b);  
System.out.println(sum);
```

With debugger

```
Pause  
  
↓  
  
Inspect variables  
  
↓  
  
Continue
```

Cleaner and faster.

---

## Example 2

```
public static int add(int a, int b) {  
    int sum = a + b;  
    return sum;  
}  
  
public static void main(String[] args) {  
    int result = add(10, 20);  
    System.out.println(result);  
}
```

Place a breakpoint inside

```
int sum = a + b;
```

Debugger pauses.

You can inspect

```
a = 10  
b = 20  
sum = not calculated yet
```

Execute one line.

Now

```
sum = 30
```

This helps verify whether your logic is correct.

---

## Where Should You Place Breakpoints?

Usually on

- Before an exception
- Before an if condition
- Inside loops
- Before database calls
- Before API calls
- Before return statements

Avoid placing breakpoints on every line.

---

## Common Mistakes

- ✗ Adding breakpoints everywhere.
  - ✗ Forgetting to remove unnecessary breakpoints.
  - ✗ Using only `System.out.println()` when a debugger is available.
- 

## Best Practice

Start with one breakpoint near the suspected bug.

If needed, move deeper into the code.

---

## Coding Exercise

Predict what you'll see.

```
public static void main(String[] args) {  
  
    int x = 5;  
  
    int y = 8;
```

```
int z = x * y;  
  
System.out.println(z);  
  
}
```

Breakpoint on

```
int z = x * y;
```

Before executing that line,

What are the variable values?

Answer

```
x = 5  
  
y = 8  
  
z = Not assigned yet
```

After stepping over,

```
z = 40
```

---

## Hard & Tricky Interview Questions

### 1. What is the difference between debugging and testing?

**Answer:**

Testing finds that a problem exists. Debugging finds the root cause and fixes it.

---

### 2. Why shouldn't you immediately add null checks to fix a `NullPointerException`?

**Answer:**

Because it hides the symptom instead of identifying why the object became null.

---

### 3. What information does a stack trace provide?

**Answer:**

- Exception type
  - Exception message
  - Call sequence
  - File names
  - Line numbers
- 

### 4. Why are breakpoints better than many `System.out.println()` statements?

**Answer:**

Breakpoints pause execution without modifying code and let you inspect variables, objects, and execution flow interactively.

---

### 5. If a stack trace contains ten method calls, where do you usually start debugging?

**Answer:**

Start with the **first application method** where the exception occurred, because that's usually the origin of the problem.

---

### 6. Can a bug disappear after restarting the application but still exist?

**Answer:**

Yes. Restarting may reset the application's state, but the underlying root cause can still be present.

---

## Interview Summary

| Topic | Key Point |
|-------|-----------|
|-------|-----------|

|             |                                                       |
|-------------|-------------------------------------------------------|
| Debugging   | Find the root cause before fixing.                    |
| Guessing    | Random code changes without evidence.                 |
| Stack Trace | Shows the exception type, message, and call sequence. |
| Breakpoint  | Pauses execution for inspection.                      |

## One-Liner Interview Revision

- **Debugging** → Finding the root cause of a problem systematically.
- **Guessing** → Changing code without understanding the issue.
- **Stack Trace** → A report showing where and why an exception occurred.
- **Exception Type** → Tells what went wrong.
- **Line Number** → Shows where to begin debugging.
- **Breakpoint** → Pauses execution so you can inspect the program state.
- **Golden Rule** → **Reproduce** → **Investigate** → **Fix** → **Verify**, not **Guess** → **Change** → **Hope**.

This is the depth I'll maintain for the remaining debugging topics: practical, interview-oriented, and detailed enough for backend interviews without becoming a textbook.